



ISLAMIC UNIVERSITY OF TECHNOLOGY

Project Name: Lengra Ant

Team Name: Powerpuff Girls

Topic Of Project: Building Application Using MATLAB/ Python

Course No: EEE 4416

Course Name: Simulation Lab

Submitted by:

Name	Student ID
Maisha Fahmida Semonty	210021208
Munshaat Atiq Nuzaba	210021210
Fabiha Anbar Orpa	210021226
Sumaiya Aziz	210021238
Tashfia Islam	210021242

Date of Submission: 01 - 06 – 2024

Abstract

This project involves the development of a MATLAB application that simulates Langton's Ant, a cellular automaton.

The objectives of this project are –

1. Implement Langton's Ant behavior on a grid.
2. To understand the underlying algorithms
3. To observe the emergent behavior from simple rules
4. Create a user-friendly interface for easy interaction.
5. Visualize the ant's movement in real-time on the grid.
6. Allow users to customize simulation parameters (grid size, number of steps).

Key findings include the emergence of complex patterns from the ant's movement and the efficiency of MATLAB's App Designer for creating interactive applications.

Introduction

For this software based , app development project, we have tried to implement a MATLAB application that allows users to observe and analyze the behavior of Langton's Ant. Langton's Ant is a well-known example of a cellular automaton. It is a theoretical two-dimensional universal Turing machine invented by Chris Langton in 1986.

Langton's Ant operates on an infinite grid of cells, where each cell can be in one of two states: black or white. The ant is initially placed on a cell and follows a set of rules as it moves across the grid:

Rule 1: If the ant is on a white cell (in our case it's pink), it turns 90 degrees to the right, flips the color of the cell to black, and moves forward one step.

Rule 2: If the ant is on a black cell, it turns 90 degrees to the left, flips the color of the cell to white, and moves forward one step.

These simple rules can lead to the emergence of complex and intricate patterns on the grid as the ant moves along its path.

Our aim was to provide an interactive platform to visualize complex and unpredictable patterns of the ant and provide an interactive way for users to explore how initial conditions affect the ant's behavior that arises from simple rules. The application allows users to specify the size of the grid and the number of steps for the simulation. It provides a user-friendly interface where users can observe the ant's movement on the grid in real-time and analyze the emergent patterns and behavior.

The application visualizes the grid using a color map, with black and pink representing the two cell states. The ant's position is updated at each step based on the current cell color and the rules of Langton's Ant. The grid visualization is continuously updated to reflect the changes in cell colors as the simulation progresses. The user can also stop, resume or refresh the ant's following movements.

Motivation: Our motivation for undertaking the project on Langton's Ant Simulation stemmed from an unexpected turn of events. Initially, our team had proposed an idea to develop an app focused on creating a period calendar. Unfortunately, our project supervisor rejected this idea, believing it lacked the potential for complexity and creative exploration.

Undeterred, our supervisor then presented us with a range of alternative ideas, one of which was Langton's Ant. We were immediately fascinated by it due to its emergent patterns and inherent complexity. The ant's behavior is influenced by the color of the grid cells it encounters, leading to the emergence of intricate patterns over time. We were intrigued to create a simulation that could simulate the intriguing patterns and behaviors that arise from simple rules.

Besides, the project presented an excellent opportunity for us to enhance our programming skills and deepen our understanding of algorithms and simulations. By implementing Langton's Ant, we could gain hands-on experience with coding, algorithm design, and problem-solving.

In summary, the rejection of our initial idea led us to discover the fascination of Langton's Ant Simulation. We were eager to challenge ourselves, deepen our programming skills, and explore the complexity that can emerge from seemingly basic rules.

Methodology:

Theory:

The methodology for the Langton's Ant simulation project involves designing a MATLAB app using App Designer to visualize the movement of Langton's Ant on a grid. The project is implemented using object-oriented programming principles, encapsulating the functionality within a class that inherits from `matlab.apps.AppBase`. The key components of the methodology include the algorithm for Langton's Ant, the design and functionality of the user interface, and the logic for controlling the simulation.

Algorithm:

Langton's Ant is a well-known cellular automaton that follows simple rules to move on a grid. The algorithm is defined as follows:

1. Initialization

- **Grid:** Initialize a 2D grid where each cell can be either black (0) or white (1).
- **Ant's Position:** Start the ant at the center of the grid.
- **Ant's Direction:** Initially set the ant's direction to "up".

- **Steps:** Define the number of steps the ant will take.

2. Main Simulation Loop

For each step, perform the following actions:

a. Check the Current Cell

- Determine the color of the cell where the ant is currently located (pink or black).

b. Turn the Ant

- If the current cell is black (0), turn the ant 90 degrees to the right.
- If the current cell is pink (1), turn the ant 90 degrees to the left.

c. Flip the Color of the Current Cell

- If the current cell is black (0), change it to pink (1).
- If the current cell is pink (1), change it to black (0).

d. Move the Ant Forward

- Move the ant one cell forward in the direction it is currently facing.

3. Edge Handling

- If the ant moves off the edge of the grid, it should wrap around to the opposite side .

Step by step explanation of the code:

1. Class Definition and Properties

Class Definition

`classdef EEE4416projectgui < matlab.apps.AppBase.` This line defines a new class named EEE4416projectgui that inherits from

matlab.apps.AppBase. This base class provides functionality for creating MATLAB apps with a graphical user interface (GUI).

Properties:

property	Access level	description
UIFigure	Public	Main figure for the GUI
UIAxes	Public	Axes for displaying the simulation
StartButton	Public	Button to start the simulation
PauseButton	Public	Button to pause the simulation
StopButton	Public	Button to stop the simulation
RefreshButton	Public	Button to refresh the simulation
GridSizeEditField	Public	Numeric field for grid size input
StepNumberEditField	Public	Numeric field for step number input
images	public	display images
grid	private	2D array representing the grid
antPos	Private	Current position of the ant
direction	Private	Current direction of the ant
gridSize	Private	Size of the grid
numSteps	Private	Number of steps for the simulation
paused	Private	Flag indicating if the simulation is paused
currentStep	private	Current step in the

		simulation
--	--	------------

Public properties correspond to the GUI components that the user interacts with. private properties are used internally to manage the state of the simulation.

2. Private Methods

A **private method** in programming is a function that is part of a class and can only be used within that class. It is not accessible from outside the class.

Main Simulation Method:

```
function LangtonsAntSimulation(app)
    if isempty(app.grid)
        app.grid = zeros(app.gridSize);
        app.antPos = [ceil(app.gridSize / 2), ceil(app.gridSize / 2)];
        app.direction = 'up';
        currentStep = 1;
    end

    for step = currentStep:app.numSteps
        if ~app.paused
            [app.grid, app.antPos, app.direction] =
app.LangtonsAntStep(app.grid, app.antPos, app.direction);
            app.VisualizeGrid(app.grid, app.UIAxes);
        end
    end
end
```

```

        pause(0.01);
    else
        app.currentStep = step;
        break;
    end
end
end
end

```

□ **Initialization:** If the grid is empty, it initializes the grid, the ant's position (center of the grid), direction (up), and current step (1).

□ **Main Loop:** Iterates from the current step to the total number of steps.

- **Update Grid:** Calls `LangtonsAntStep` to update the grid, ant's position, and direction.
- **Visualize Grid:** Calls `VisualizeGrid` to display the grid.
- **Pause:** If the simulation is paused, it stores the current step and breaks the loop.

Step Simulation Method:

```

function [newGrid, newPos, newDirection] = LangtonsAntStep(app,
grid, pos, direction)

```

```

    if grid(pos(1), pos(2)) == 0

```

```

        newDirection = app.turnRight(direction);

```

```

        grid(pos(1), pos(2)) = 1;

```

```

    else

```

```

    newDirection = app.turnLeft(direction);

    grid(pos(1), pos(2)) = 0;

end

newPos = app.moveForward(pos, newDirection, app.gridSize);

newGrid = grid;

end

```

- **Cell State:** Checks the current cell's state.
 - **White Cell:** Turns right and changes the cell to black.
 - **Black Cell:** Turns left and changes the cell to white.
- **Move Ant:** Updates the ant's position based on the new direction.

Direction Handling Methods:

```

function newDirection = turnRight(app, direction)

switch direction

case 'up'

    newDirection = 'right';

case 'right'

    newDirection = 'down';

case 'down'

    newDirection = 'left';

```

```
    case 'left'
        newDirection = 'up';
    end
end
```

```
function newDirection = turnLeft(app, direction)
```

```
    switch direction
        case 'up'
            newDirection = 'left';
        case 'right'
            newDirection = 'up';
        case 'down'
            newDirection = 'right';
        case 'left'
            newDirection = 'down';
        end
    end
```

- ☐ **turnRight:** Changes the direction clockwise.
- ☐ **turnLeft:** Changes the direction counterclockwise.

Movement Method:

```
function newPos = moveForward(app, pos, direction, gridSize)

    switch direction

        case 'up'

            newPos = [pos(1) - 1, pos(2)];

        case 'right'

            newPos = [pos(1), pos(2) + 1];

        case 'down'

            newPos = [pos(1) + 1, pos(2)];

        case 'left'

            newPos = [pos(1), pos(2) - 1];

    end

    newPos = mod(newPos - 1, gridSize) + 1;

end
```

- **Direction Handling:** Moves the ant in the current direction.
- **Boundary Handling:** Ensures the ant stays within the grid bounds using modulo operation.

Visualization Method:

```
function VisualizeGrid(app, grid, ax)
```

```
    color1 = [0, 0, 0]; %black
```

```
    color2 = [1, 0.3, 0.7]; %pink
```

```
    axis(ax,'off');
```

```
    imagesc(ax, grid);
```

```
    colormap(ax, [color1; color2]);
```

```
    axis(ax, 'square');
```

```
end
```

□ **Colors:** Defines colors for black and pink cells.

□ **Display:** Uses imagesc to display the grid and sets the colormap.

Control Methods:

```
function PauseSimulation(app)
```

```
    app.paused = true;
```

```
    app.PauseButton.Text = 'Resume';
```

```
end
```

```
function ResumeSimulation(app)
```

```
    app.paused = false;
```

```
    app.PauseButton.Text = 'Pause';
```

```
    app.LangtonsAntSimulation();
```

end

```
function StopSimulation(app)
```

```
    app.paused = true;
```

end

```
function RefreshApp(app)
```

```
    app.UIFigure.Visible = 'off';
```

```
    app.UIFigure.Visible = 'on';
```

end

□ **PauseSimulation:** Pauses the simulation and changes the button text to 'Resume'.

□ **ResumeSimulation:** Resumes the simulation and changes the button text to 'Pause'.

□ **StopSimulation:** Stops the simulation and resets the pause button text.

□ **RefreshApp:** Resets the simulation state and refreshes the UI.

These methods update the 'paused' flag and UI as needed.

3. Component Callbacks

StartButtonPushed:

```
function StartButtonPushed(app, event)
```

```

app.gridSize = app.GridSizeEditField.Value;
app.numSteps = app.StepNumberEditField.Value;
app.paused = false;
app.grid = []; % Reset the grid to start fresh
app.LangtonsAntSimulation();
end

```

This method starts the simulation:

- Reads the grid size and number of steps from the edit fields.
- Sets the paused flag to false.
- Resets the grid to start fresh.
- Calls the LangtonsAntSimulation method to start the simulation.

PauseButtonPushed:

```

function PauseButtonPushed(app, event)
if app.paused
    % If paused, resume the simulation
    app.paused = false;
    app.PauseButton.Text = 'Pause';
    app.LangtonsAntSimulation();
else
    % If running, pause the simulation
    app.paused = true;
    app.PauseButton.Text = 'Resume';
end
end

```

This method toggles between pausing and resuming the simulation:

- Checks the paused flag and updates it accordingly.
- Updates the pause button text.
- Resumes the simulation if it was paused.

StopButtonPushed:

```
function StopButtonPushed(app, event)
```

```
    app.paused=true;
```

```
end
```

Stop Simulation: Calls ‘StopSimulation’.

RefreshButtonPushed:

```
function RefreshButtonPushed(app, event)
```

```
    app.grid = [];
```

```
    app.antPos = [];
```

```
    app.direction = [];
```

```
    app.paused = true;
```

```
    cla(app.UIAxes);
```

```
    app.UIFigure.Visible = 'off';
```

```
    app.UIFigure.Visible = 'on';
```

```
end
```

Refresh App: Calls ‘RefreshApp’.

These callbacks are linked to UI buttons to control the simulation.

4. Component Initialization

Create Components:

methods (Access = private)

```
function createComponents(app)

    app.UIFigure = uifigure('Visible', 'off');

    app.UIFigure.Color = [0.5686 0.5255 0.5255];

    app.UIFigure.Position = [100 100 865 551];

    app.UIFigure.Name = 'MATLAB App';


    app.UIAxes = uiaxes(app.UIFigure);

    title(app.UIAxes, 'Ant Movement')

    app.UIAxes.FontName = 'Century Gothic';

    app.UIAxes.FontWeight = 'bold';

    app.UIAxes.XColor = 'none';

    app.UIAxes.XTick = [];

    app.UIAxes.YColor = 'none';

    app.UIAxes.YTick = [];

    app.UIAxes.ZColor = 'none';
```

```

app.UIAxes.FontSize = 32;

app.UIAxes.Position = [251 241 367 238];


app.StartButton = uibutton(app.UIFigure, 'push');

app.StartButton.ButtonPushedFcn = createCallbackFcn(app,
@StartButtonPushed, true);

app.StartButton.FontName = 'Baskerville Old Face';

app.StartButton.FontSize = 20;

app.StartButton.FontWeight = 'bold';

app.StartButton.Position = [505 175 100 32];

app.StartButton.Text = {'Start'; ''};


app.GridSizeEditFieldLabel = uilabel(app.UIFigure);

app.GridSizeEditFieldLabel.HorizontalAlignment = 'right';

app.GridSizeEditFieldLabel.FontName = 'BankGothic Lt BT';

app.GridSizeEditFieldLabel.FontSize = 28;

app.GridSizeEditFieldLabel.FontWeight = 'bold';

app.GridSizeEditFieldLabel.Position

```

Public Methods:

These methods are responsible for the construction and deletion of the app.

Constructor Method:

```
% Construct app

function app = lengra_ant

    % Create UIFigure and components

    createComponents(app)

    % Register the app with App Designer

    registerApp(app, app.UIFigure)

    if nargin == 0

        clear app

    end

end
```

This is the constructor method for the `lengra_ant` class. It is called when an instance of the class is created.

1.Creating UIFigure and Components:

```
createComponents(app)
```

This line calls the `createComponents` method, which sets up the UI for the app, including creating the figure and adding all the UI components like buttons, images, axes, etc.

2.Registering the App:

```
registerApp(app, app.UIFigure)
```

This line registers the app with App Designer, which is necessary for handling interactions and managing the app lifecycle.

3.Clearing the App Variable:

```
if nargout == 0
```

```
    clear app
```

```
end
```

This conditional statement checks if there are any output arguments. If not, it clears the app variable to ensure that it does not stay in the workspace unnecessarily. This helps in managing memory and keeping the workspace clean.

Deletion Method:

```
% Code that executes before app deletion
```

```
function delete(app)
```

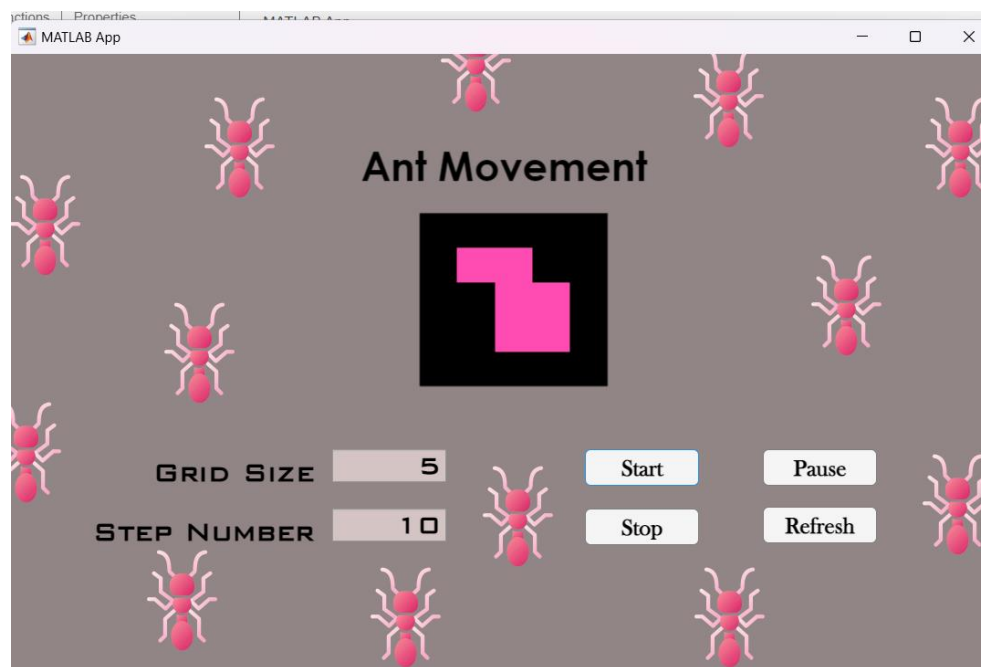
```
    % Delete UIFigure when app is deleted
```

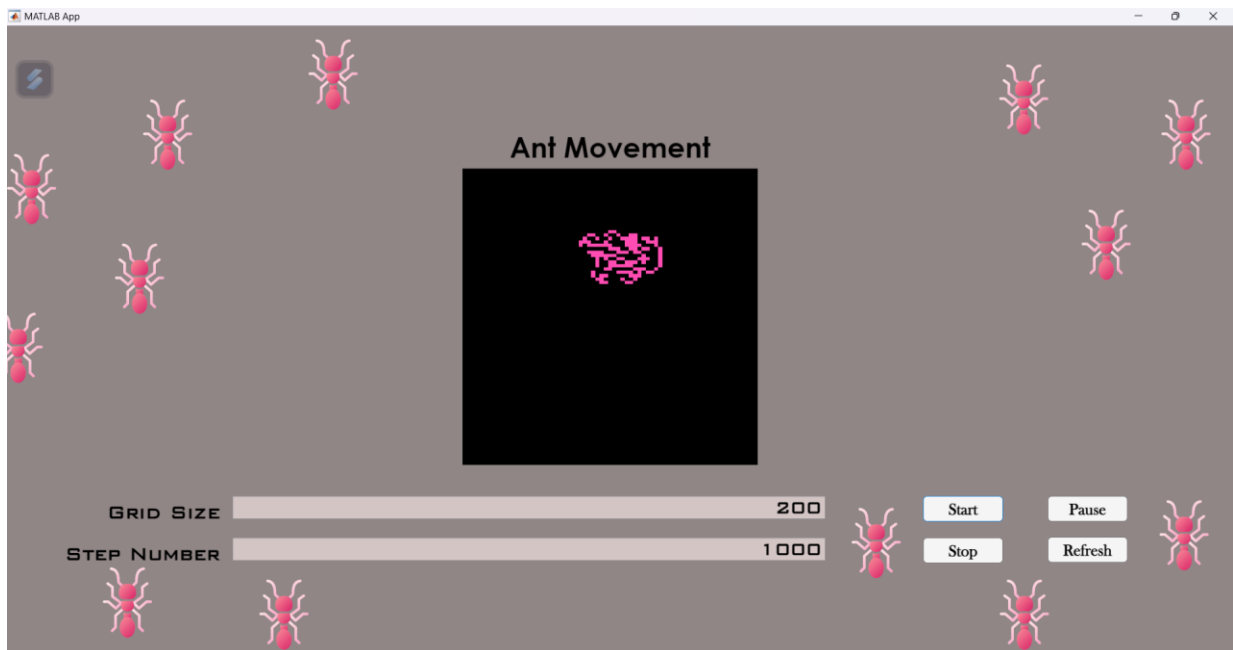
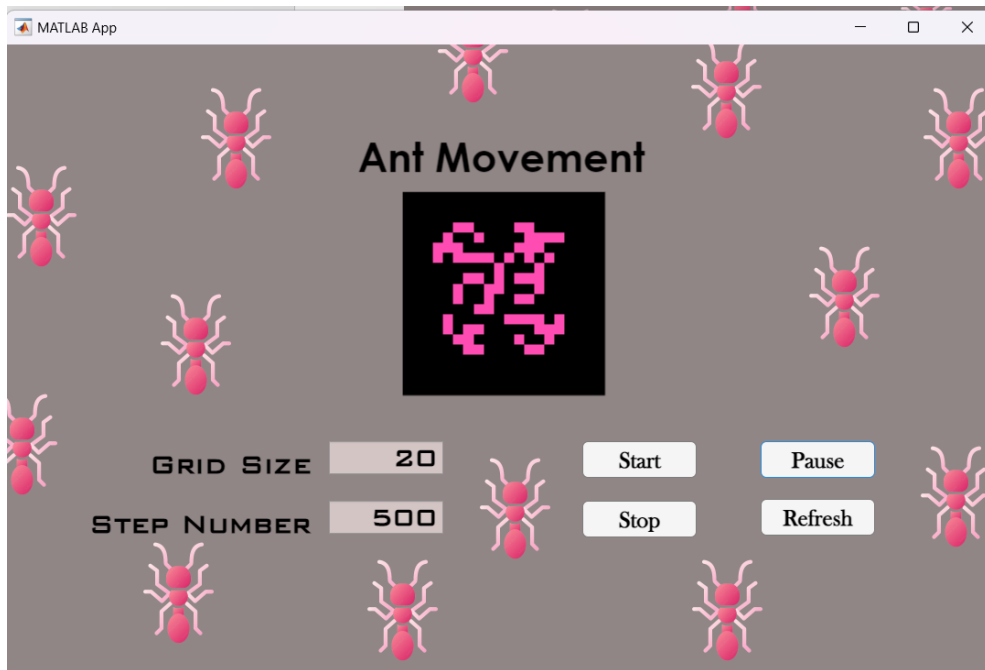
```
    delete(app.UIFigure)
```

```
end
```

This is the destructor method for the `lengra_ant` class. It is called when an instance of the class is deleted.

Sample Output:





Contribution Table:

Student ID	Contribution(%)	Specific Contribution
210021208	19.5	Coding(public method), Debugging, Modification, Report writing (Methodology, Algorithm, part by part explanation)
210021210	20.5	Coding - properties, Simulation method, Button function (pause and resume), Debugging, Primary layout of the app
210021226	19.7	Coding (Direction and Movement logic), Simulation, Algorithm designing ,Report Writing(Algorithm, part by part explanation)
210021238	20	Coding(component initialization), Debugging, Modification, Report writing (Theory, Abstract, Functions,

		Algorithm explanation)
210021242	20.3	Coding(visualization method, control method, component callbacks),Graphical design enhancement of the app layout, Debugging(stop and refresh button)

Conclusion:

The Langoton's ant project successfully implements a MATLAB application for simulating Langton's Ant behavior. By visualizing the ant's movement on a grid, the project provides an interactive and educational tool for studying complex systems and emergent behavior.

This project significantly enhanced our MATLAB coding skills and deepened our understanding of various functionalities. Implementing Langton's Ant algorithm improved our problem-solving abilities and proficiency in matrix manipulation. Using MATLAB's App Designer, we learned to create interactive GUIs with buttons, edit fields, and axes, along with writing callback functions for user interactions.

We gained experience with MATLAB functions and techniques like matrix indexing, colormap manipulation, and animation control. Visualizing the ant's movement on a grid taught us how to use the 'imagesc' function and customize colormaps, improving our ability to present data graphically. The contributions of each group member have been vital in the successful completion of this project.